

Chapter 15

Generative adversarial networks

A *generative adversarial network* or *GAN* is an unsupervised model that aims to generate new samples that are indistinguishable from a set of training examples. GANs are just mechanisms to create new samples; they do not build a probability distribution over the modeled data and hence cannot evaluate the probability that a new data point belongs to the same distribution.

In a GAN, the main *generator* network creates samples by mapping random noise to the output data space. If a second *discriminator* network cannot distinguish between the generated samples and the real examples, the samples must be plausible. If this network *can* tell the difference, this provides a training signal that can be fed back to improve the quality of the samples. This idea is simple, but training GANs is difficult: the learning algorithm can be unstable, and although GANs may learn to generate realistic samples, this does not imply that they learn to generate *all* possible samples.

GANs have been applied to many types of data, including audio, 3D models, text, video, and graphs. However, they have found the most success in the image domain, where they can produce samples that are almost indistinguishable from real pictures. Accordingly, the examples in this chapter focus on synthesizing images.

15.1 Discrimination as a signal

We aim to generate new samples $\{\mathbf{x}_j^*\}$ that are drawn from the same distribution as a set of real training data $\{\mathbf{x}_i\}$. A single new sample $\mathbf{x}_j^*$ is generated by (i) choosing a *latent variable* $\mathbf{z}_j$ from a simple base distribution (e.g., a standard normal) and then (ii) passing this data through a network $\mathbf{x}_j^* = \mathbf{g}[\mathbf{z}_j, \boldsymbol{\theta}]$ with parameters $\boldsymbol{\theta}$. This network is known as the *generator*. During the learning process, the goal is to find parameters $\boldsymbol{\theta}$ so that the samples $\{\mathbf{x}_j^*\}$ look “similar” to the real data $\{\mathbf{x}_i\}$ (see figure 14.2a).

Similarity can be defined in many ways, but the GAN uses the principle that the samples should be statistically indistinguishable from the true data. To this end, a second network $f[\bullet, \phi]$ with parameters ϕ called the *discriminator* is introduced. This network aims to classify its input as being a real example or a generated sample. If this

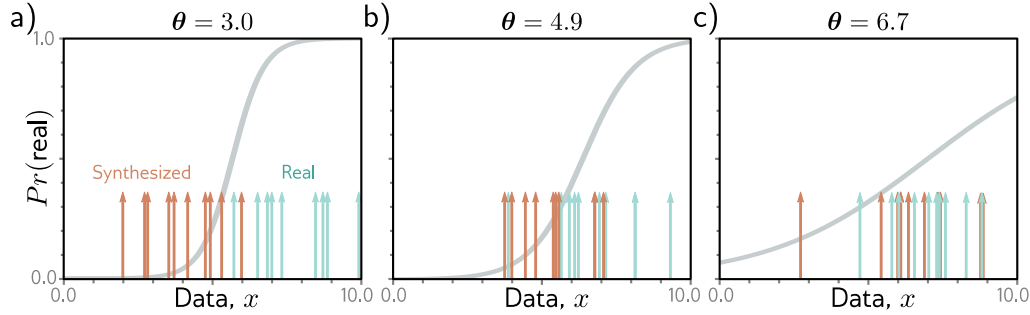


Figure 15.1 GAN mechanism. a) Given a parameterized function (a generator) that synthesizes samples (orange arrows) and a batch of real examples (cyan arrows), we train a discriminator to distinguish the real examples from the generated samples (sigmoid curve indicates the estimated probability that the data point is real). b) The generator is trained by modifying its parameters so that the discriminator becomes less confident the samples were synthetic (in this case, by moving the orange samples to the right). The discriminator is then updated. c) Alternating updates to the generator and discriminator cause the generated samples to become indistinguishable from real examples and the impetus to change the generator (i.e., the slope of the sigmoid function) to diminish.

proves impossible, the generated samples are indistinguishable from the real examples, and we have succeeded. If it is possible, the discriminator provides a signal that can be used to improve the generation process.

Figure 15.1 illustrates this scheme. We start with a training set $\{x_i\}$ of real 1D examples. A different batch of ten of these examples $\{x_i\}_{i=1}^{10}$ is shown in each panel (cyan arrows). To create a batch of samples $\{x_j^*\}$, we use the simple generator:

$$x_j^* = g[z_j, \theta] = z_j + \theta, \quad (15.1)$$

where latent variables $\{z_j\}$ are drawn from a standard normal distribution, and the parameter θ translates the generated samples along the x-axis (figure 15.1).

At initialization, $\theta = 3.0$, and the generated samples (orange arrows) lie to the left of the real examples (cyan arrows). The discriminator is trained to distinguish the generated samples from the real examples (the sigmoid curve indicates the probability that a data point is real). During training, the generator parameters θ are manipulated to increase the probability that its samples are classified as real. Here, this means increasing θ so that the samples move rightwards where the sigmoid curve is higher.

We alternate between updating the discriminator and the generator. Figures 15.1b–c show two iterations of this process. It gradually becomes harder to classify the data, so the impetus to change θ becomes weaker (i.e., the sigmoid becomes flatter). At the end of the process, there is no way to distinguish the two sets of data; the discriminator, which now has chance performance, is discarded, and we are left with a generator that makes plausible samples.

Notebook 15.1
GAN toy example

15.1.1 GAN loss function

We now define the loss function for training GANs more precisely. The discriminator $f[\mathbf{x}, \phi]$ takes input $\mathbf{x}$, has parameters ϕ , and returns a scalar that is higher when it believes the input is a real example. This is a binary classification task, so we adapt the binary cross-entropy loss function (section 5.4), which originally had the form:

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}} \left[\sum_i -(1 - y_i) \log [1 - \operatorname{sig}[f[\mathbf{x}_i, \phi]]] - y_i \log [\operatorname{sig}[f[\mathbf{x}_i, \phi]]] \right], \quad (15.2)$$

where $y_i \in \{0, 1\}$ is the label, and $\operatorname{sig}[\bullet]$ is the logistic sigmoid function (figure 5.7).

In this case, we assume that the real examples $\mathbf{x}$ have label $y = 1$ and the generated samples $\mathbf{x}^*$ have label $y = 0$ so that:

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}} \left[\sum_j -\log [1 - \operatorname{sig}[f[\mathbf{x}_j^*, \phi]]] - \sum_i \log [\operatorname{sig}[f[\mathbf{x}_i, \phi]]] \right], \quad (15.3)$$

where i and j index the real examples and generated samples, respectively.

Now we substitute the definition for the generator $\mathbf{x}_j^* = \mathbf{g}[\mathbf{z}_j, \theta]$ and note that we must maximize with respect to θ since we want the generated samples to be misclassified (i.e., have low likelihood of being synthetic or high negative log-likelihood):

$$\hat{\theta} = \underset{\theta}{\operatorname{argmax}} \left[\min_{\phi} \left[\sum_j -\log [1 - \operatorname{sig}[f[\mathbf{g}[\mathbf{z}_j, \theta], \phi]]] - \sum_i \log [\operatorname{sig}[f[\mathbf{x}_i, \phi]]] \right] \right]. \quad (15.4)$$

15.1.2 Training GANs

Equation 15.4 is a more complex loss function than we have seen before; the discriminator parameters ϕ are manipulated to minimize the loss function, and the generative parameters θ are manipulated to maximize the loss function. GAN training is characterized as a *minimax game*; the generator tries to find new ways to fool the discriminator, which in turn searches for new ways to distinguish generated samples from real examples. Technically, the solution is a *Nash equilibrium* — the optimization algorithm searches for a position that is simultaneously a minimum of one function and a maximum of the other. If training proceeds as planned, then upon convergence, $\mathbf{g}[\mathbf{z}, \theta]$ will be drawn from the same distribution as the data, and $\operatorname{sig}[f[\bullet, \phi]]$ will be at chance (i.e., 0.5).

To train the GAN, we can divide equation 15.4 into two loss functions:

$$\begin{aligned} L[\phi] &= \sum_j -\log [1 - \operatorname{sig}[f[\mathbf{g}[\mathbf{z}_j, \theta], \phi]]] - \sum_i \log [\operatorname{sig}[f[\mathbf{x}_i, \phi]]] \\ L[\theta] &= \sum_j \log [1 - \operatorname{sig}[f[\mathbf{g}[\mathbf{z}_j, \theta], \phi]]], \end{aligned} \quad (15.5)$$

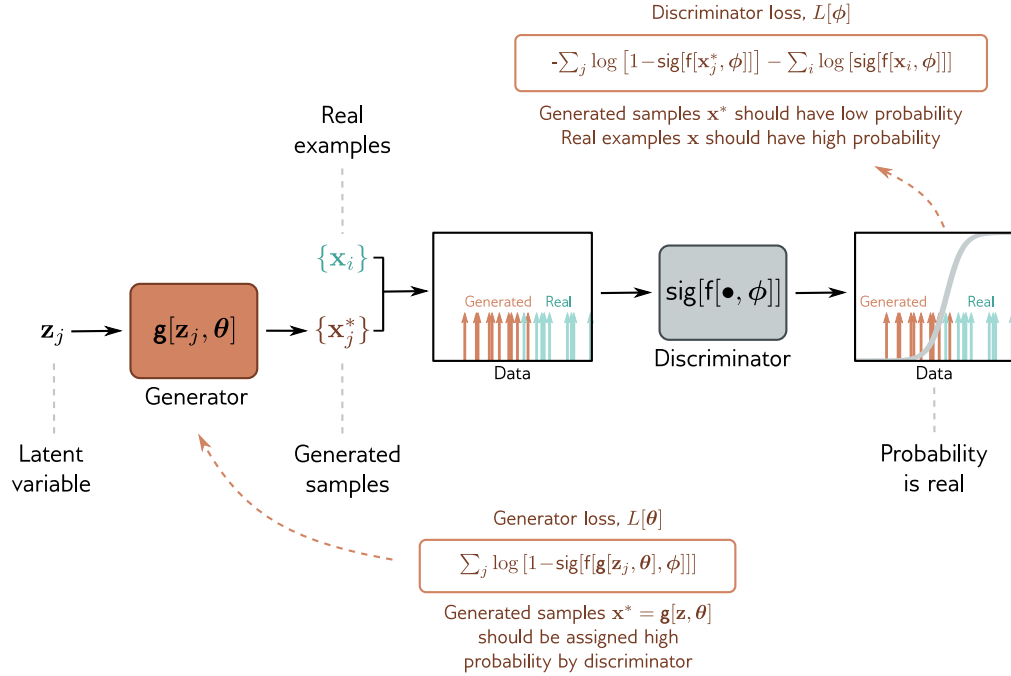


Figure 15.2 GAN loss functions. A latent variable z_j is drawn from the base distribution and passed through the generator to create a sample x^* . A batch $\{x_j^*\}$ of samples and a batch of real examples $\{x_i\}$ are passed to the discriminator, which assigns a probability that each is real. The discriminator parameters ϕ are modified to assign high probability to the real examples and low probability to the generated samples. The generator parameters θ are modified to “fool” the discriminator into assigning the generated samples a high probability.

where we multiplied the second function by minus one to convert to a minimization problem and dropped the second term, which has no dependence on θ . Minimizing the first loss function trains the discriminator. Minimizing the second trains the generator.

At each step, we draw a batch of latent variables z_j from the base distribution and pass these through the generator to create samples $x_j^* = g[z_j, \theta]$. Then we choose a batch of real training examples x_i . Given the two batches, we can now perform one or more gradient descent steps on each loss function (figure 15.2).

15.1.3 Deep convolutional GAN

The *deep convolutional GAN* or *DCGAN* was an early GAN architecture specialized for generating images (figure 15.3). The input to the generator $g[z, \theta]$ is a 100D latent

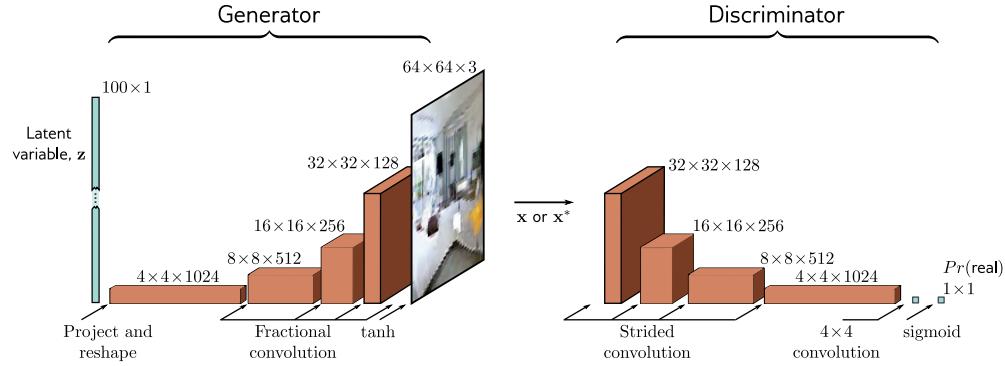


Figure 15.3 DCGAN architecture. In the generator, a 100D latent variable $\mathbf{z}$ is drawn from a uniform distribution and mapped by a linear transformation to a 4×4 representation with 1024 channels. This is then passed through a series of convolutional layers that gradually upsample the representation and decrease the number of channels. At the end is a tanh function that maps the $64 \times 64 \times 3$ representation to a fixed range so that it can represent an image. The discriminator consists of a standard convolutional net that classifies the input as either a real example or a generated sample.

variable $\mathbf{z}$ sampled from a uniform distribution. This is then mapped to a 4×4 spatial representation with 1024 channels using a linear transformation. Four convolutional layers follow, each of which uses a fractionally-strided convolution that doubles the resolution (i.e., a convolution with a stride of 0.5). At the final layer, the $64 \times 64 \times 3$ signal is passed through a tanh function to generate an image $\mathbf{x}^*$ in the range $[-1, 1]$. The discriminator $f[\bullet, \phi]$ is a standard convolutional network where the final convolutional layer reduces the size to 1×1 with one channel. This single number is passed through a sigmoid function $\text{sig}[\bullet]$ to create the output probability.

After training, the discriminator is discarded. To create new samples, latent variables $\mathbf{z}$ are drawn from the base distribution and passed through the generator. Example results are shown in figure 15.4.

15.1.4 Difficulty training GANs

Theoretically, the GAN is fairly straightforward. However, GANs are notoriously difficult to train. For example, to get the DCGAN to train reliably, it was necessary to (i) use strided convolutions for upsampling and downsampling; (ii) use BatchNorm in both generator and discriminator except in the last and first layers, respectively; (iii) use the leaky ReLU activation function (figure 3.13) in the discriminator; and (iv) use the Adam optimizer but with a lower momentum coefficient than usual. This is unusual. Most deep learning models are relatively robust to such choices.



Figure 15.4 Synthesized images from the DCGAN model. a) Random samples drawn from DCGAN trained on a faces dataset. b) Random samples using the ImageNet database (see figure 10.15). c) Random samples drawn from the LSUN scene understanding dataset. Adapted from Radford et al. (2015).

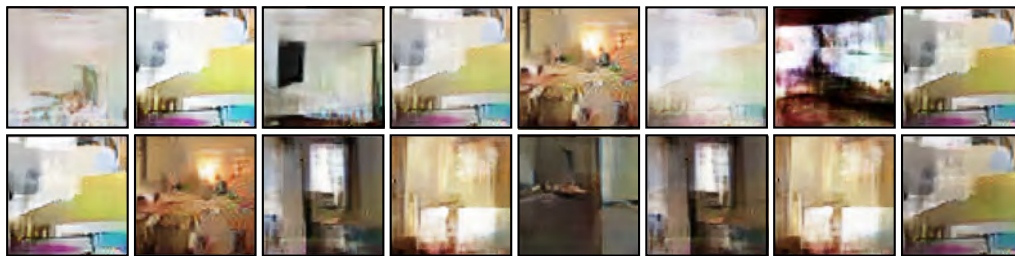


Figure 15.5 Mode collapse. Synthesized images from a GAN trained on the LSUN scene understanding dataset using an MLP generator with a similar number of parameters and layers to the DCGAN. The samples are low quality, and many are similar. Adapted from Arjovsky et al. (2017).

A common failure mode is that the generator makes plausible samples, but these only represent a subset of the data (e.g., for faces, it might never generate faces with beards). This is known as *mode dropping*. An extreme version of this phenomenon can occur where the generator entirely or mostly ignores the latent variables $\mathbf{z}$ and collapses all samples to one or a few points; this is known as *mode collapse* (figure 15.5).

15.2 Improving stability

To understand *why* GANs are difficult to train, it's necessary to understand exactly *what* the loss function represents.

15.2.1 Analysis of GAN loss function

If we divide the two sums in the first line of equation 15.5 by the numbers I, J of real and generated samples, then the loss function can be written in terms of expectations:

$$\begin{aligned} L[\phi] &= -\frac{1}{J} \sum_{j=1}^J \left(\log [1 - \text{sig}[f[\mathbf{x}_j^*, \phi]]] \right) - \frac{1}{I} \sum_{i=1}^I \left(\log [\text{sig}[f[\mathbf{x}_i, \phi]]] \right) \quad (15.6) \\ &\approx -\mathbb{E}_{\mathbf{x}^*} \left[\log [1 - \text{sig}[f[\mathbf{x}^*, \phi]]] \right] - \mathbb{E}_{\mathbf{x}} \left[\log [\text{sig}[f[\mathbf{x}, \phi]]] \right] \\ &= -\int Pr(\mathbf{x}^*) \log [1 - \text{sig}[f[\mathbf{x}^*, \phi]]] d\mathbf{x}^* - \int Pr(\mathbf{x}) \log [\text{sig}[f[\mathbf{x}, \phi]]] d\mathbf{x}, \end{aligned}$$

where $Pr(\mathbf{x}^*)$ is the probability distribution over the generated samples, and $Pr(\mathbf{x})$ is the true probability distribution over the real examples.

When $I = J$, the optimal discriminator for an example $\tilde{\mathbf{x}}$ of unknown origin is:

$$Pr(\text{real}|\tilde{\mathbf{x}}) = \text{sig}[f[\tilde{\mathbf{x}}, \phi]] = \frac{Pr(\tilde{\mathbf{x}}|\text{real})}{Pr(\tilde{\mathbf{x}}|\text{generated}) + Pr(\tilde{\mathbf{x}}|\text{real})} = \frac{Pr(\mathbf{x})}{Pr(\mathbf{x}^*) + Pr(\mathbf{x})}, \quad (15.7)$$

where on the right hand side, we evaluate $\tilde{\mathbf{x}}$ against the generated distribution $Pr(\mathbf{x}^*)$ and the real distribution $Pr(\mathbf{x})$. Substituting into equation 15.6, we get:

$$\begin{aligned} L[\phi] &= -\int Pr(\mathbf{x}^*) \log [1 - \text{sig}[f[\mathbf{x}^*, \phi]]] d\mathbf{x}^* - \int Pr(\mathbf{x}) \log [\text{sig}[f[\mathbf{x}, \phi]]] d\mathbf{x} \quad (15.8) \\ &= -\int Pr(\mathbf{x}^*) \log \left[1 - \frac{Pr(\mathbf{x})}{Pr(\mathbf{x}^*) + Pr(\mathbf{x})} \right] d\mathbf{x}^* - \int Pr(\mathbf{x}) \log \left[\frac{Pr(\mathbf{x})}{Pr(\mathbf{x}^*) + Pr(\mathbf{x})} \right] d\mathbf{x} \\ &= -\int Pr(\mathbf{x}^*) \log \left[\frac{Pr(\mathbf{x}^*)}{Pr(\mathbf{x}^*) + Pr(\mathbf{x})} \right] d\mathbf{x}^* - \int Pr(\mathbf{x}) \log \left[\frac{Pr(\mathbf{x})}{Pr(\mathbf{x}^*) + Pr(\mathbf{x})} \right] d\mathbf{x}. \end{aligned}$$

Problems 15.1–15.2

Appendix C.5.2
Jensen-Shannon
divergence

Disregarding additive and multiplicative constants, this is the **Jensen-Shannon divergence** between the synthesized distribution $Pr(\mathbf{x}^*)$ and the true distribution $Pr(\mathbf{x})$:

$$\begin{aligned} D_{JS} [Pr(\mathbf{x}^*) || Pr(\mathbf{x})] &\quad (15.9) \\ &= \frac{1}{2} D_{KL} \left[Pr(\mathbf{x}^*) \left\| \frac{Pr(\mathbf{x}^*) + Pr(\mathbf{x})}{2} \right\| \right] + \frac{1}{2} D_{KL} \left[Pr(\mathbf{x}) \left\| \frac{Pr(\mathbf{x}^*) + Pr(\mathbf{x})}{2} \right\| \right] \\ &= \frac{1}{2} \int \underbrace{Pr(\mathbf{x}^*) \log \left[\frac{2Pr(\mathbf{x}^*)}{Pr(\mathbf{x}^*) + Pr(\mathbf{x})} \right] d\mathbf{x}^*}_{\text{quality}} + \frac{1}{2} \int \underbrace{Pr(\mathbf{x}) \log \left[\frac{2Pr(\mathbf{x})}{Pr(\mathbf{x}^*) + Pr(\mathbf{x})} \right] d\mathbf{x}}_{\text{coverage}}. \end{aligned}$$

where $D_{KL}[\bullet||\bullet]$ is the **Kullback-Leibler divergence**.

The first term indicates the distance will be small if, wherever the sample density $Pr(\mathbf{x}^*)$ is high, the mixture $(Pr(\mathbf{x}^*) + Pr(\mathbf{x}))/2$ has high probability. In other words, it penalizes regions with samples $\mathbf{x}^*$ but no real examples $\mathbf{x}$; it enforces *quality*. The second term says that the distance will be small if, wherever the true density $Pr(\mathbf{x})$

Appendix C.5.1
Kullback-Leibler
divergence

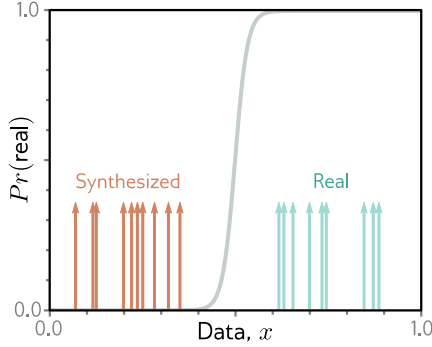


Figure 15.6 Problem with GAN loss function. If the generated samples (orange arrows) are easy to distinguish from the real examples (cyan arrows), then the discriminator (sigmoid) may have a very shallow slope at the positions of the samples; hence, the gradient to update the parameter of the generator may be tiny.

is high, the mixture $(Pr(\mathbf{x}^*) + Pr(\mathbf{x}))/2$ has high probability. In other words, it penalizes regions with real examples but no samples. It enforces *coverage*. Referring to equation 15.6, we see that the second term does not depend on the generator, which consequently doesn't care about coverage; it is happy to generate a subset of possible examples accurately. This is the putative reason for mode dropping.

15.2.2 Vanishing gradients

In the previous section, we saw that when the discriminator is optimal, the loss function maximizes a measure of the distance between the generated and real samples. However, there is a potential problem with using this distance between probability distributions as the criterion for optimizing GANs. If the probability distributions are completely disjoint, this distance is maximal, and any small change to the generator will not decrease the loss. The same phenomenon can be seen when we consider the original formulation; if the discriminator can perfectly separate the generated and real samples, no small change to the generated data will change the classification score (figure 15.6).

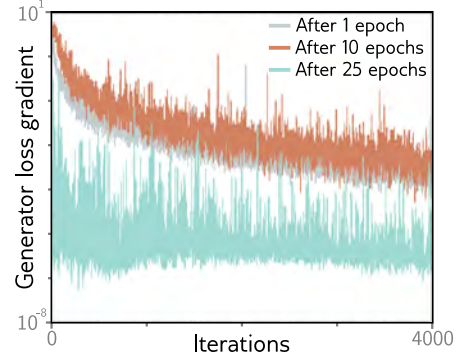
Unfortunately, the distributions of generated samples and real examples may *really* be disjoint; the generated samples lie in a subspace that is the size of the latent variable $\mathbf{z}$, and the real examples also lie in a low-dimensional subspace due to the physical processes that created the data (figure 1.9). There may be little or no overlap between these subspaces, and the result is very small or no gradients.

Figure 15.7 provides empirical evidence to support this hypothesis. If the DCGAN generator is frozen and the discriminator is updated repeatedly so that its classification performance improves, the generator gradients decrease. In short, there is a very fine balance between the quality of the discriminator and the generator; if the discriminator becomes too good, the training updates of the generator are attenuated.

15.2.3 Wasserstein distance

The previous sections showed that (i) the GAN loss can be interpreted in terms of distances between probability distributions and that (ii) the gradient of this distance

Figure 15.7 Vanishing gradients in the generator of a DCGAN. The generator is frozen after 1, 10, and 25 epochs, and the discriminator is trained further. The gradient of the generator decreases rapidly (note log scale); if the discriminator becomes too accurate, the gradients for the generator vanish. Adapted from Arjovsky & Bottou (2017).



becomes zero when the generated samples are too easy to distinguish from the real examples. The obvious way forward is to choose a distance metric with better properties.

The *Wasserstein* or (for discrete distributions) *earth mover's* distance is the quantity of work required to transport the probability mass from one distribution to create the other. Here, “work” is defined as the mass multiplied by the distance moved. This immediately sounds more promising; the Wasserstein distance is well-defined even when the distributions are disjoint and decreases smoothly as they become closer to one another.

15.2.4 Wasserstein distance for discrete distributions

The Wasserstein distance is easiest to understand for discrete distributions (figure 15.8). Consider distributions $Pr(x = i)$ and $q(x = j)$ defined over K bins. Assume there is a cost C_{ij} associated with moving one unit of mass from bin i in the first distribution to bin j in the second; this cost might be the absolute difference $|i - j|$ between the indices. The amounts that are moved form the *transport plan* and are stored in a matrix $\mathbf{P}$.

The Wasserstein distance is defined as:

$$D_w[Pr(x)||q(x)] = \min_{\mathbf{P}} \left[\sum_{i,j} P_{ij} \cdot |i - j| \right], \quad (15.10)$$

subject to the constraints that:

$$\begin{aligned} \sum_j P_{ij} &= Pr(x = i) && \text{initial distribution of } Pr(x) \\ \sum_i P_{ij} &= q(x = j) && \text{initial distribution of } q(x) \\ P_{ij} &\geq 0 && \text{non-negative masses.} \end{aligned} \quad (15.11)$$

In other words, the Wasserstein distance is the solution to a constrained minimization problem that maps the mass of one distribution to the other. This is inconvenient as we must solve this minimization problem over the elements P_{ij} every time we want to compute the distance. Fortunately, this is a standard problem that is easily solved for small systems of equations. It is a *linear programming problem* in its *primal form*:

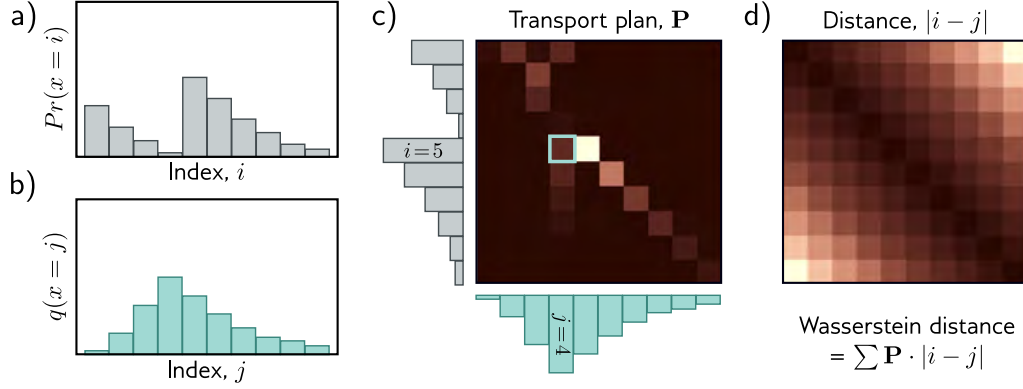


Figure 15.8 Wasserstein or earth mover's distance. a) Consider the discrete distribution $Pr(x = i)$. b) We wish to move the probability mass to create the target distribution $q(x = j)$. c) The transport plan $\mathbf{P}$ identifies how much mass will be moved from i to j . For example, the cyan highlighted square p_{54} indicates how much mass will be moved from $i = 5$ to $j = 4$. The elements of the transport plan must be non-negative, the sum over j must be $Pr(x = i)$, and the sum over i must be $q(x = j)$. Hence $\mathbf{P}$ is a joint probability distribution. d) The distance matrix between elements i and j . The optimal transport plan $\mathbf{P}$ minimizes the sum of the pointwise product of $\mathbf{P}$ and the distance matrix (termed the Wasserstein distance). Hence, the elements of $\mathbf{P}$ tend to lie close to the diagonal where the distance cost is lowest. Adapted from Hermann (2017).

primal form		dual form	
minimize	$\mathbf{c}^T \mathbf{p}$,	maximize	$\mathbf{b}^T \mathbf{f}$,
such that	$\mathbf{A} \mathbf{p} = \mathbf{b}$	such that	$\mathbf{A}^T \mathbf{f} \leq \mathbf{c}$
and	$\mathbf{p} \geq \mathbf{0}$		

where $\mathbf{p}$ contains the vectorized elements P_{ij} that determine the amount of mass moved, $\mathbf{c}$ contains the distances, $\mathbf{A} \mathbf{p} = \mathbf{b}$ contains the initial distribution constraints, and $\mathbf{p} \geq \mathbf{0}$ ensures the masses moved are non-negative.¹

As for all linear programming problems, there is an equivalent *dual problem* with the same solution. Here, we maximize with respect to a variable $\mathbf{f}$ that is applied to the initial distributions, subject to constraints that depend on the distances $\mathbf{c}$. The solution to this dual problem is:

$$D_w[Pr(x)||q(x)] = \max_{\mathbf{f}} \left[\sum_i Pr(x=i) f_i - \sum_j q(x=j) f_j \right], \quad (15.12)$$

¹The mathematical background is omitted due to space constraints. Linear programming is a standard problem with well-known algorithms for finding the minimum.

Problem 15.3

Notebook 15.2
Wasserstein
distance

subject to the constraint that:

$$|f_{i+1} - f_i| < 1. \quad (15.13)$$

In other words, we optimize over a new set of variables $\{f_i\}$ where adjacent values cannot change by more than one.

15.2.5 Wasserstein distance for continuous distributions

Translating these results back to the continuous multi-dimensional domain, the equivalent of the primal form (equation 15.10) is:

Problems 15.4–15.5

$$D_w[Pr(\mathbf{x}), q(\mathbf{x})] = \min_{\pi[\bullet, \bullet]} \left[\int \int \pi(\mathbf{x}_1, \mathbf{x}_2) \cdot \|\mathbf{x}_1 - \mathbf{x}_2\| d\mathbf{x}_1 d\mathbf{x}_2 \right], \quad (15.14)$$

subject to constraints similar to equation 15.11 on the transport plan $\pi(\mathbf{x}_1, \mathbf{x}_2)$ representing the mass moved from position $\mathbf{x}_1$ to $\mathbf{x}_2$. The equivalent of the dual form (equation 15.12) is:

$$D_w[Pr(\mathbf{x}), q(\mathbf{x})] = \max_{f[\mathbf{x}]} \left[\int Pr(\mathbf{x}) f[\mathbf{x}] d\mathbf{x} - \int q(\mathbf{x}) f[\mathbf{x}] d\mathbf{x} \right], \quad (15.15)$$

Appendix B.1.1
Lipschitz constant

subject to the constraint that the [Lipschitz constant](#) of the function $f[\mathbf{x}]$ is less than one (i.e., the absolute gradient of the function is less than one).

15.2.6 Wasserstein GAN loss function

In the context of neural networks, we maximize over the space of functions $f[\mathbf{x}]$ by optimizing the parameters ϕ in a neural network $f[\mathbf{x}, \phi]$, and we approximate these integrals using generated samples $\mathbf{x}_j^*$ and real examples $\mathbf{x}_i$:

$$\begin{aligned} L[\phi] &= \sum_j f[\mathbf{x}_j^*, \phi] - \sum_i f[\mathbf{x}_i, \phi] \\ &= \sum_j f[\mathbf{g}[\mathbf{z}_j, \theta], \phi] - \sum_i f[\mathbf{x}_i, \phi], \end{aligned} \quad (15.16)$$

where we must constrain the neural network discriminator $f[\mathbf{x}_i, \phi]$ to have an absolute gradient norm of less than one at every position $\mathbf{x}$:

$$\left| \frac{\partial f[\mathbf{x}, \phi]}{\partial \mathbf{x}} \right| < 1. \quad (15.17)$$

One way to achieve this is to clip the discriminator weights to a small range (e.g., ± 0.01). An alternative is the *gradient penalty Wasserstein GAN* or *WGAN-GP*, which adds a regularization term that increases as the gradient norm deviates from unity.

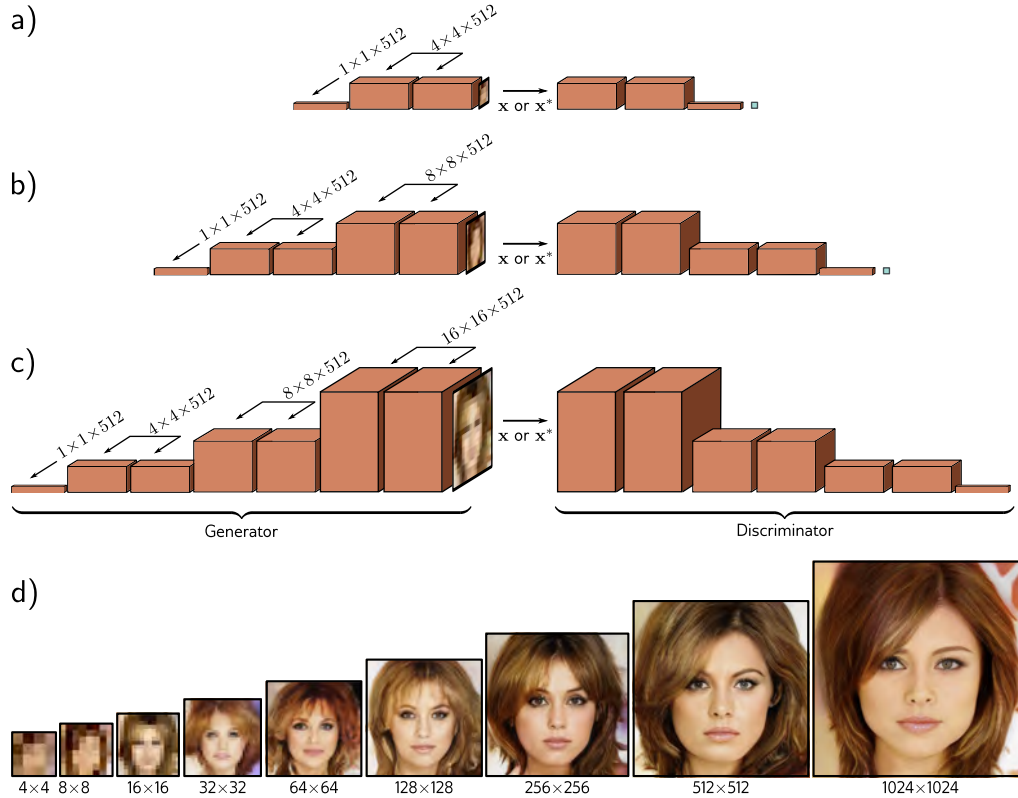


Figure 15.9 Progressive growing. a) The generator is initially trained to create very small 4×4 images, and the discriminator to identify if these images are synthesized or downsampled real images. b) After training at this low-resolution terminates, subsequent layers are added to the generator to generate 8×8 images. Similar layers are added to the discriminator to downsample back again. c) This process continues to create 16×16 images and so on. In this way, a GAN that produces very realistic high-resolution images can be trained. d) Images of increasing resolution generated at different stages from the same latent variable. Adapted from Wolf (2021), using method of Karras et al. (2018).

15.3 Progressive growing, minibatch discrimination, and truncation

The Wasserstein formulation makes GAN training more stable. However, further machinery is needed to generate high-quality images. We now review *progressive growing*, *minibatch discrimination*, and *truncation*, which all improve output quality.

In *progressive growing* (figure 15.9), we first train a GAN that synthesizes 4×4 images using an architecture similar to the DCGAN. Then we add subsequent layers to the generator, which upsample the representation and perform further processing to create

Figure 15.10 Truncation. The quality of GAN samples can be traded off against diversity by rejecting samples from the latent variable $\mathbf{z}$ that fall further than τ standard deviations from the mean. a) If this threshold is large ($\tau = 2.0$), the samples are visually varied but may have defects. b–c) As this threshold is decreased, the average visual quality improves, but the diversity decreases. d) With a very small threshold, the samples look almost identical. By judiciously choosing this threshold, it's possible to increase the average quality of GAN results. Adapted from Brock et al. (2019).

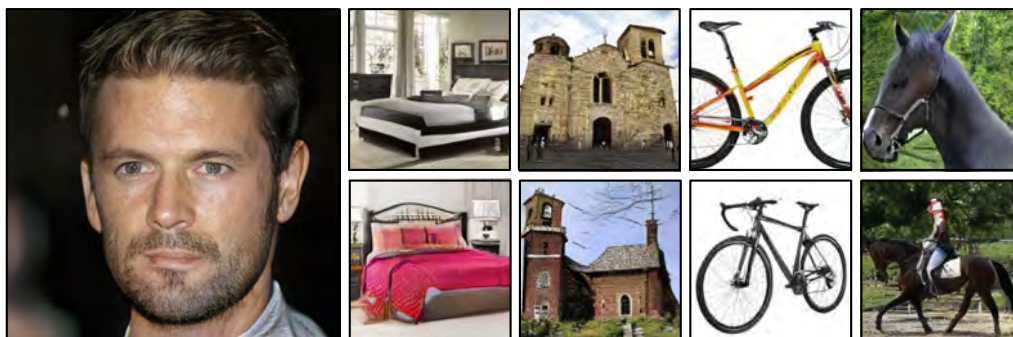
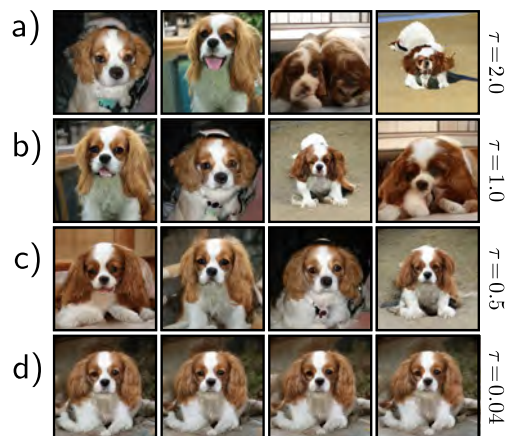


Figure 15.11 Combining methods. GANs can generate realistic images of faces when trained on CELEBA-HQ dataset and more complex, variable objects when trained on LSUN categories. Adapted from Karras et al. (2018).



Figure 15.12 Traversing latent space of progressive GAN trained on LSUN cars. Moving in the latent space produces car images that change smoothly. This usually only works for short trajectories; eventually, the latent variable moves to somewhere that produces unrealistic images. Adapted from Karras et al. (2018).

an 8×8 image. The discriminator also has extra layers added to it so that it can receive the higher-resolution images and classify them as either being generated samples or real examples. In practice, the higher-resolution layers gradually “fade in” over time; initially, the higher-resolution image is an upsampled version of the previous result, passed via a residual connection, and the new layers gradually take over.

Mini-batch discrimination ensures that the samples have sufficient variety and hence helps prevent mode collapse. This can be done by computing feature statistics across the mini-batches of synthesized and real data. These can be summarized and added as a feature map (usually toward the end of the discriminator). This allows the discriminator to send a signal back to the generator, encouraging it to include a similar amount of variation in the synthesized data as in the original dataset.

Another trick to improve generation results is *truncation* (figure 15.10), in which only latent variables $\mathbf{z}$ with high probability (i.e., close to the mean) are chosen during sampling. This reduces the variation in the samples but improves their quality. Careful normalization and regularization schemes also improve sample quality. Using combinations of these methods, GANs can synthesize varied and realistic images (figure 15.11). Moving smoothly through the latent space can also sometimes produce realistic interpolations from one synthesized image to another (figure 15.12).

Problem 15.6

15.4 Conditional generation

GANs produce realistic images but don’t specify their attributes: we can’t choose the hair color, ethnicity, or age of faces, without training separate GANs for each combination of characteristics. *Conditional generation* models provide us with this control.

15.4.1 Conditional GAN

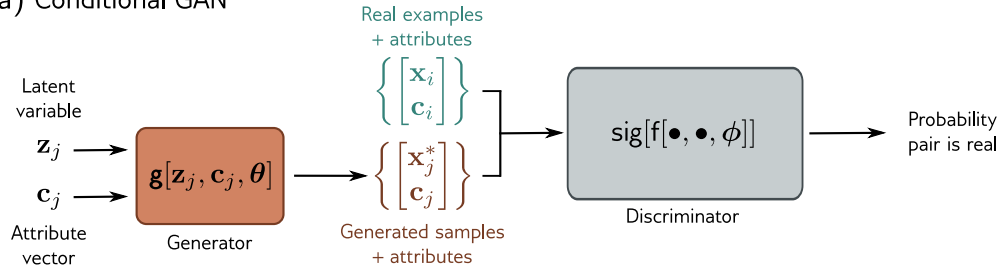
The *conditional GAN* passes a vector $\mathbf{c}$ of attributes to both the generator and discriminator, which are now written as $\mathbf{g}[\mathbf{z}, \mathbf{c}, \boldsymbol{\theta}]$ and $\mathbf{f}[\mathbf{x}, \mathbf{c}, \boldsymbol{\phi}]$, respectively. The generator aims to transform the latent variable $\mathbf{z}$ into a data sample $\mathbf{x}$ with the correct attribute $\mathbf{c}$. The discriminator’s goal is to distinguish between (i) the generated sample with the target attribute or (ii) a real example with the real attribute (figure 15.13a).

For the generator, the attribute $\mathbf{c}$ can be appended to the latent vector $\mathbf{z}$. For the discriminator, it may be appended to the input if the data are 1D. If the data comprise images, the attribute can be linearly transformed to a 2D representation and appended as an extra channel to the discriminator input or to one of its intermediate hidden layers.

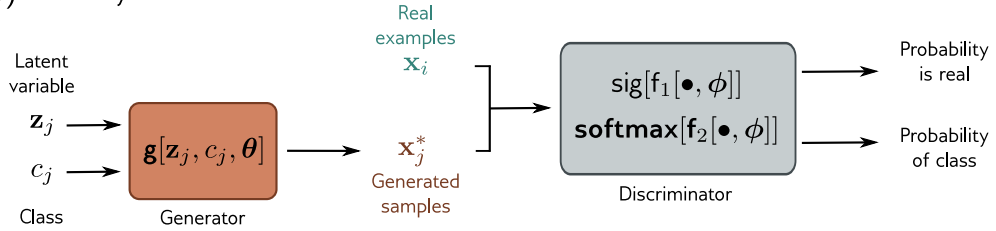
15.4.2 Auxiliary classifier GAN

The *auxiliary classifier GAN* or *ACGAN* simplifies conditional generation by requiring that the discriminator correctly predicts the attribute (figure 15.13b). For a discrete

a) Conditional GAN



b) Auxiliary classifier GAN



c) InfoGAN

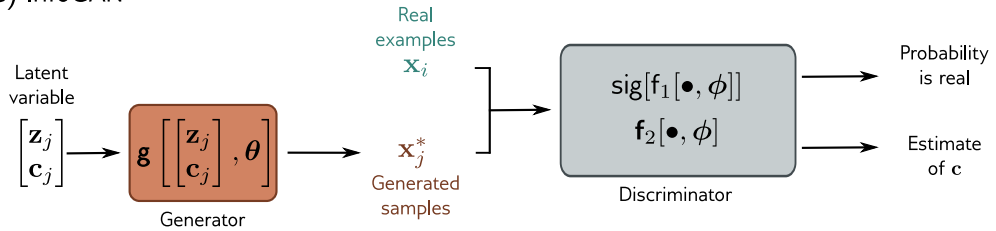


Figure 15.13 Conditional generation. a) The generator of the conditional GAN also receives an attribute vector $\mathbf{c}$ describing some aspect of the image. As usual, the discriminator receives either a real example or a generated sample, but now it also receives the attribute vector; this encourages the samples both to be realistic and compatible with the attribute. b) The generator of the auxiliary classifier GAN (ACGAN) takes a discrete attribute variable. The discriminator must both (i) determine if its input is real or synthetic and (ii) identify the class correctly. c) The InfoGAN splits the latent variable into noise $\mathbf{z}$ and unspecified random attributes $\mathbf{c}$. The discriminator must distinguish if its input is real and also reconstruct these attributes. In practice, this means that the variables $\mathbf{c}$ correspond to salient aspects of the data with real-world interpretations (i.e., the latent space is *disentangled*).



Figure 15.14 Auxiliary classifier GAN. The generator takes a class label as well as the latent vector. The discriminator must both identify if the data point is real *and* predict the class label. This model was trained on ten ImageNet classes. Left to right: generated examples of monarch butterflies, goldfinches, daisies, redshanks, and gray whales. Adapted from Odena et al. (2017).

attribute with C categories, the discriminator takes the real/synthesized image as input and has $C + 1$ outputs; the first is passed through a sigmoid function and predicts if the sample is generated or real. The remaining outputs are passed through a softmax function to predict the probability that the data belongs to each of the C classes. Networks trained with this method can synthesize multiple classes from ImageNet (figure 15.14).

15.4.3 InfoGAN

The conditional GAN and ACGAN both generate samples that have predetermined attributes. By contrast, InfoGAN (figure 15.13c) attempts to identify important attributes automatically. The generator takes a vector consisting of random noise variables $\mathbf{z}$ and *random* attribute variables $\mathbf{c}$. The discriminator both predicts whether the image is real or synthesized and estimates the attribute variables.

The insight is that interpretable real-world characteristics should be easiest to predict and hence will be represented in the attribute variables $\mathbf{c}$. The attributes in $\mathbf{c}$ may be discrete (and a binary or multiclass cross-entropy loss would be used) or continuous (and a least squares loss would be used). The discrete variables identify categories in the data, and the continuous ones identify gradual modes of variation (figure 15.15).

15.5 Image translation

Although the adversarial discriminator was first used in the context of the GAN for generating random samples, it can also be used as a prior that favors realism in tasks that translate one data example into another. This is most commonly done with images,

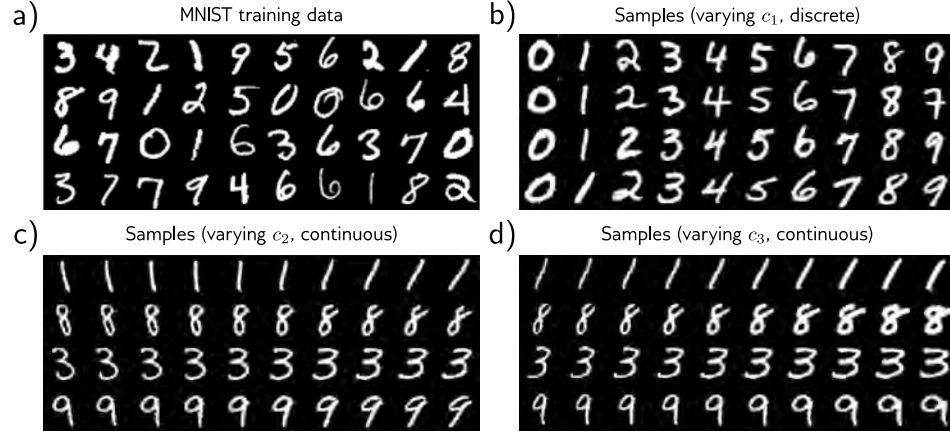


Figure 15.15 InfoGAN for MNIST. a) Training examples from the MNIST database, which consists of 28×28 pixel images of handwritten digits. b) The first attribute c_1 is categorical with 10 categories; each column shows samples generated with one of these categories. The InfoGAN recovers the ten digits. The attribute vectors c_2 and c_3 are continuous. c) Moving from left to right, each column represents a different value of c_2 while keeping the other latent variables constant. This attribute seems to correspond to the orientation of the character. d) The third attribute seems to correspond to the thickness of the stroke. Adapted from Chen et al. (2016b).

where we might want to translate a grayscale image to color, a noisy image to a clean one, a blurry image to a sharp one, or a sketch to a photo-realistic image.

This section discusses three image translation models that use different amounts of manual labeling. The Pix2Pix model uses before/after pairs for training. Models with adversarial losses use before/after pairs for the main model but also exploit unpaired “after” images in the discriminator. The CycleGAN model uses unpaired images.

15.5.1 Pix2Pix

The Pix2Pix model (figure 15.16) is a network $\hat{\mathbf{x}} = \mathbf{g}[\mathbf{c}, \boldsymbol{\theta}]$ that maps one image $\mathbf{c}$ to a different style image $\mathbf{x}$ using a U-Net (figure 11.10) with parameters $\boldsymbol{\theta}$. A typical use case would be colorization, where the input $\mathbf{c}$ is grayscale, and the output $\mathbf{g}[\mathbf{c}, \boldsymbol{\theta}]$ is color. The output should be similar to the input, and this is encouraged using a *content loss* that penalizes the ℓ_1 norm $\|\mathbf{x} - \mathbf{g}[\mathbf{c}, \boldsymbol{\theta}]\|_1$ between the input $\mathbf{c}$ and ground truth output $\mathbf{x}$.

However, the output image should also look like a realistic conversion of the input. This is encouraged by using an adversarial discriminator $f[\mathbf{c}, \mathbf{x}, \phi]$, which ingests the before and after images $\mathbf{c}$ and $\mathbf{x}$. At each step, the discriminator tries to distinguish between a real before/after pair and a before/synthesized pair. To the extent that these

can be distinguished successfully, a feedback signal is provided to modify the U-Net to make its output more realistic. Since the content loss ensures that the large-scale image structure is correct, the discriminator is mainly needed to ensure that the local texture is plausible. To this end, the *PatchGAN* loss is based on a purely convolutional classifier. At the last layer, each hidden unit indicates whether the region within its receptive field is real or synthesized. These responses are averaged to provide the final output.

One way to think of this model is that it is a conditional GAN where the U-Net is the generator and is conditioned on an image rather than a label. Notice, though, that the U-Net input does not include noise and so is not really a “generator” in the conventional sense. Interestingly, the original authors experimented with adding noise $\mathbf{z}$ to the U-Net in addition to the input image $\mathbf{c}$. However, the network just learned to ignore it.

15.5.2 Adversarial loss

The discriminator of the Pix2Pix model attempted to distinguish whether before/after pairs in an image translation task were plausible. This has the disadvantage that we need ground truth before/after pairs to exploit the discriminator loss. Fortunately, there is a simpler way to exploit the power of adversarial discriminators in the context of supervised learning without the need for additional labeled training data.

An *adversarial loss* adds a penalty if a discriminator can distinguish the output of a supervised network from a real example from its output domain. Accordingly, the supervised model changes its predictions to decrease this penalty. This may be done at the scale of the entire output or at the level of patches, as in the Pix2Pix algorithm. This helps improve the *realism* of complex structured outputs. However, it doesn’t necessarily lead to a better solution in terms of the original loss function.

The *super-resolution GAN* or *SRGAN* uses this approach (figure 15.17). The main model consists of a convolutional network with residual connections that ingests a low-resolution image and converts this via upsampling layers to a high-resolution image. The network is trained with three losses. The content loss measures the squared difference between the output and the true high-resolution image. The *VGG loss* or *perceptual loss* passes the synthesized and ground truth outputs through the VGG network and measures the squared difference between their activations. This encourages the image to be semantically similar to the target. Finally, the adversarial loss uses a discriminator that attempts to distinguish whether this is a real high-resolution image or an upsampled one. This encourages the output to be indistinguishable from real examples.

15.5.3 CycleGAN

The adversarial loss assumes that we have labeled before/after images for the main supervised network. The *CycleGAN* addresses the situation where we have two sets of data with distinct styles but *no* matching pairs. An example is converting a photo to the artistic style of Monet. There exist many photos and many Monet paintings, but no correspondence between them. CycleGAN exploits the idea that converting an image in

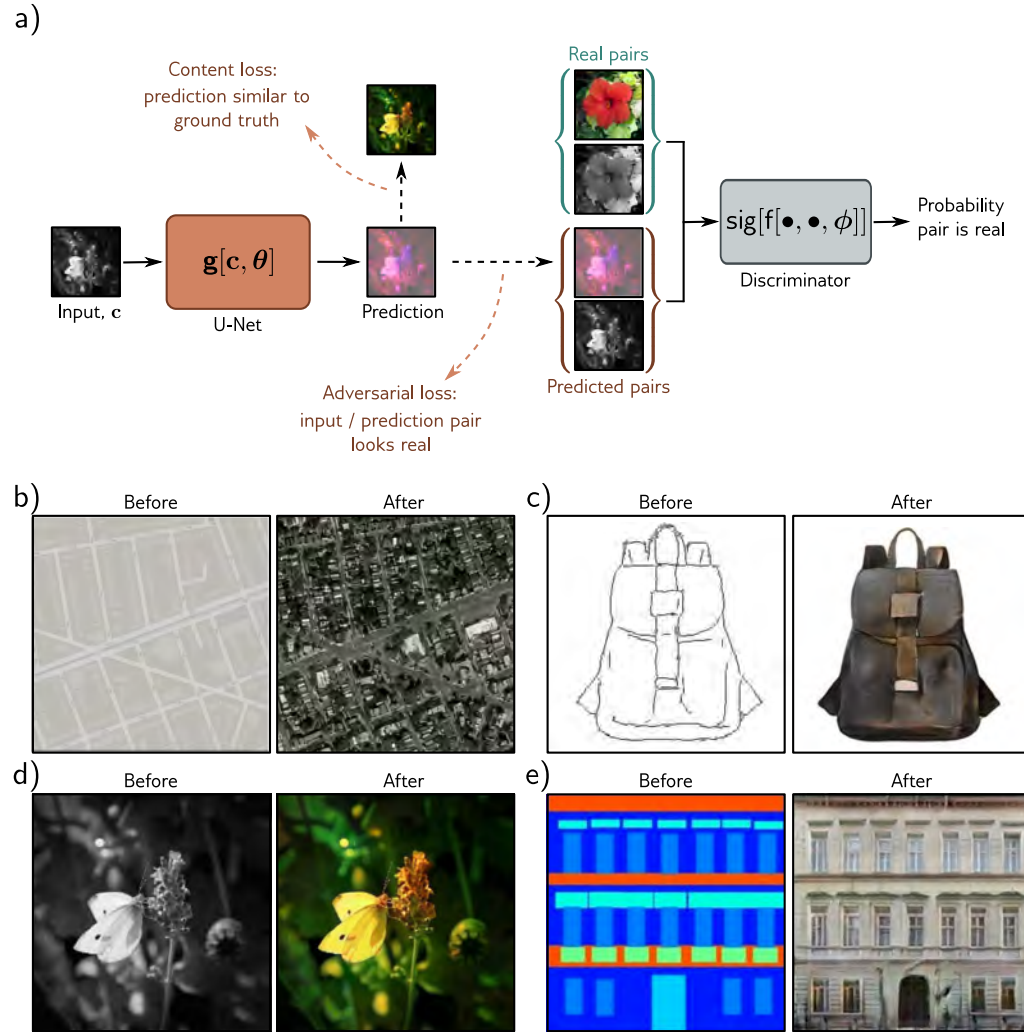


Figure 15.16 Pix2Pix model. a) The model translates an input image to a prediction in a different style using a U-Net (see figure 11.10). In this case, it maps a grayscale image to a plausibly colored version. The U-Net is trained with two losses. First, the content loss encourages the output image to have a similar structure to the input image. Second, the adversarial loss encourages the grayscale/color image pair to be indistinguishable from a real pair in each local region of these images. This framework can be adapted to many tasks, including b) translating maps to satellite imagery, c) converting sketches of bags to photorealistic examples, d) colorization, and e) converting label maps to photorealistic building facades. Adapted from Isola et al. (2017).

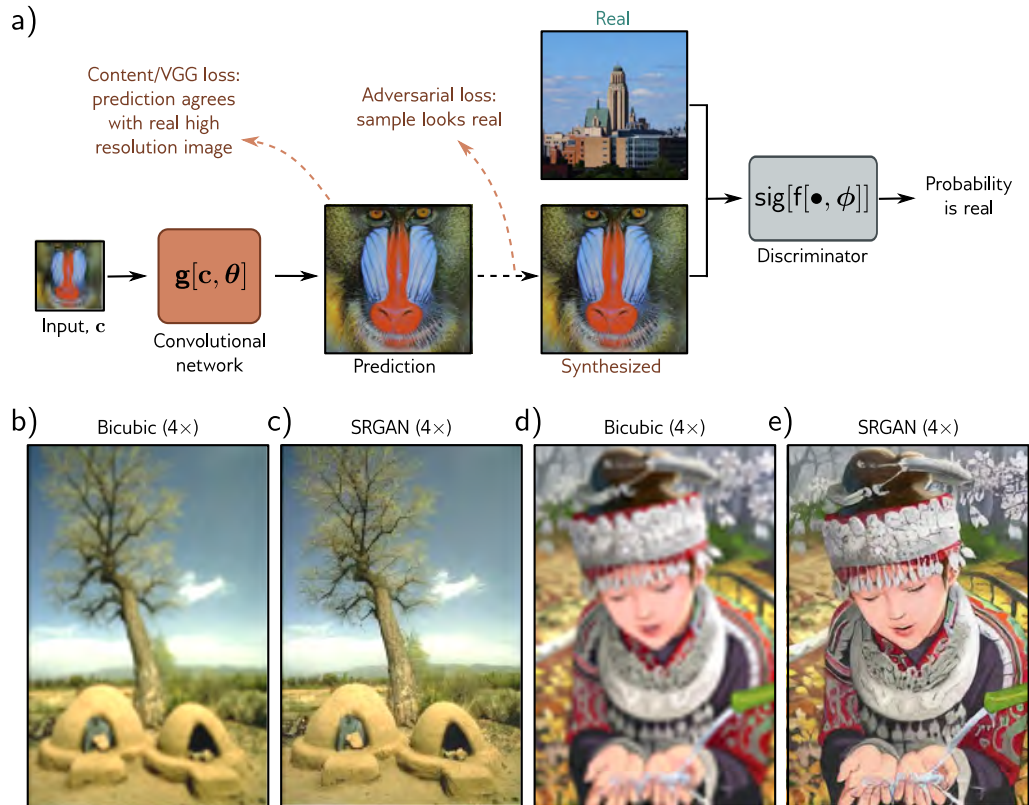


Figure 15.17 Super-resolution generative adversarial network (SRGAN). a) A convolutional network with residual connections is trained to increase the resolution of images by a factor of four. The model has losses that encourage the content to be close to the true high-resolution image. However, it also includes an adversarial loss, which penalizes results that can be distinguished from real high-resolution images. b) Upsampled image using bicubic interpolation. c) Upsampled image using SRGAN. d) Upsampled image using bicubic interpolation. e) Upsampled image using SRGAN. Adapted from Ledig et al. (2017).

one direction (e.g., photo→Monet) and then back again should recover the original.

The CycleGAN loss function is a weighted sum of three losses (figure 15.18). The content loss encourages the before and after images to be similar and is based on the ℓ_1 norm. The adversarial loss uses a discriminator to encourage the output to be indistinguishable from real examples of the target domain. Finally, the *cycle-consistency* loss encourages the mapping to be reversible. Here, two models are trained together. One maps from the first domain to the second, and the other in the opposite direction. The cycle-consistency loss will be low if the translated image can be itself translated successfully back to the image in the original domain. The model combines these three losses to train networks to translate images from one style to another and back again.

Appendix B.3.2
 ℓ_1 norm

15.6 StyleGAN

StyleGAN is a more contemporary GAN that partitions the variation in a dataset into meaningful components, each of which is controlled by a subset of the latent variables. In particular, StyleGAN controls the output image at different scales and separates style from noise. For face images, large-scale changes include face shape and head pose, medium-scale changes include the shape and details of facial features, and fine-scale changes include hair and skin color. The style components represent aspects of the image that are salient to human beings, and the noise aspects represent unimportant variation such as the exact placement of hairs, stubble, freckles, or skin pores.

The GANs that we have seen until now started from a latent variable $\mathbf{z}$ which is drawn from a standard base distribution. This was passed through a series of convolutional layers to produce the output image. However, the latent variable inputs to the generator can (i) be introduced at various points in the architecture and (ii) modify the current representation at these points in different ways. StyleGAN makes these choices judiciously to control scale and to separate style from noise (figure 15.19).

The main generative branch of StyleGAN starts with a learned constant 4×4 representation with 512 channels. This passes through a series of convolutional layers that gradually upsample the representation to generate the image at its final resolution. Two sets of random latent variables representing style and noise are introduced at each scale; the closer that they are to the output, the finer scale details they represent.

The latent variables that represent noise are independently sampled Gaussian vectors $\mathbf{z}_1, \mathbf{z}_2 \dots$ and are injected additively after each convolution operation in the main generative pipeline. They are the same spatial size as the main representation at the point that they are added but are multiplied by learned per-channel scaling factors $\psi_1, \psi_2 \dots$ and so contribute in different amounts to each channel. As the resolution of the network increases, this noise contributes at finer scales.

The latent variables that represent style begin as a $1 \times 1 \times 512$ noise tensor, which is passed through a seven-layer fully connected network to create an intermediate variable $\mathbf{w}$. This allows the network to decorrelate aspects of style so that each dimension of $\mathbf{w}$ can represent an independent real-world factor such as head pose or hair color. This variable $\mathbf{w}$ is linearly transformed to a $2 \times 1 \times 512$ tensor $\mathbf{y}$, which is used to set the per-channel mean and variance of the representation across spatial positions in the

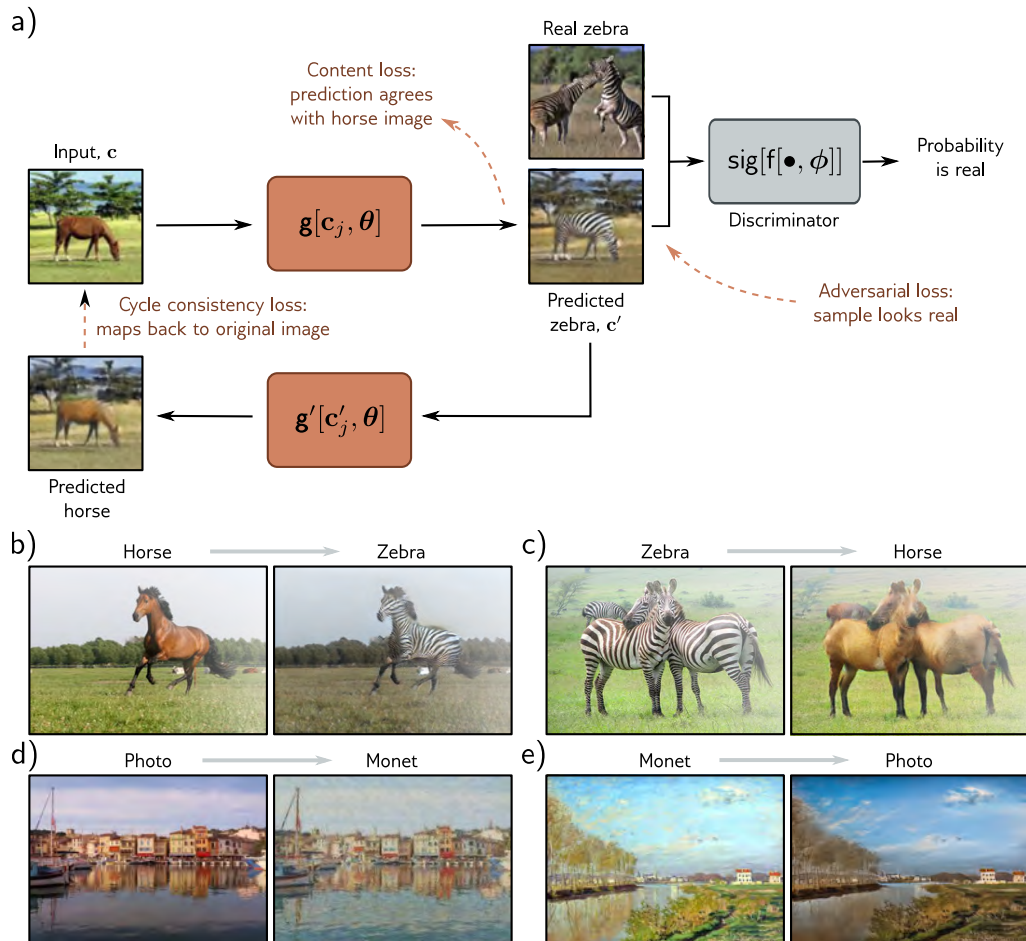


Figure 15.18 CycleGAN. Two models are trained simultaneously. The first $c' = g[c_j, \theta]$ translates from an image c in the first style (horse) to an image c' in the second style (zebra). The second model $c = g'[c'_j, \theta]$ learns the opposite mapping. The cycle consistency loss penalizes both models if they cannot successfully convert an image to the other domain and back to the original. In addition, two adversarial losses encourage the translated images to look like realistic examples of the target domain (shown here for zebra only). Two content losses encourage the details and layout of the images before and after each mapping to be similar (i.e., the zebra is in the same position and pose that the horse was and against the same background and vice versa). Adapted from Zhu et al. (2017).

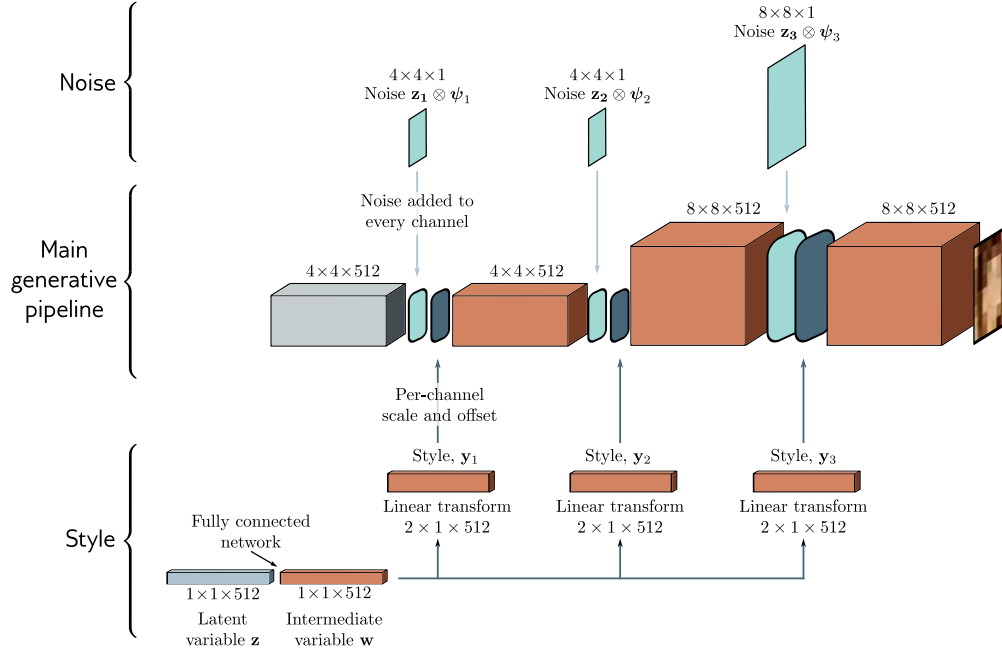


Figure 15.19 StyleGAN. The main pipeline (center row) starts with a constant learned representation (gray box). This is passed through a series of convolutional layers and gradually upsampled to create the output. Noise (top row) is added at different scales by periodically adding Gaussian variables $\mathbf{z}_\bullet$ with per-channel scaling $\psi_\bullet$. The Gaussian style variable $\mathbf{z}$ is passed through a fully connected network to create intermediate variable $\mathbf{w}$ (bottom row). This is used to set the mean and variance of each channel at various points in the pipeline.

main branch after noise addition. This is termed *adaptive instance normalization* (figure 11.14e). A series of vectors $\mathbf{y}_1, \mathbf{y}_2, \dots$ are injected in this way at several different points in the main branch, so the same style contributes at different scales. Figure 15.20 shows examples of manipulating the style and noise vectors at different scales.

15.7 Summary

GANs learn a generator network that transforms random noise into data that is indistinguishable from a training set. To this end, the generator is trained using a discriminator network that tries to distinguish real examples from generated samples. The generator is then updated so that the data that it creates is identified as being more “real” by the discriminator. The original formulation of this idea has the flaw that the training signal is weak when it’s easy to determine if the samples are real or generated. This led to the

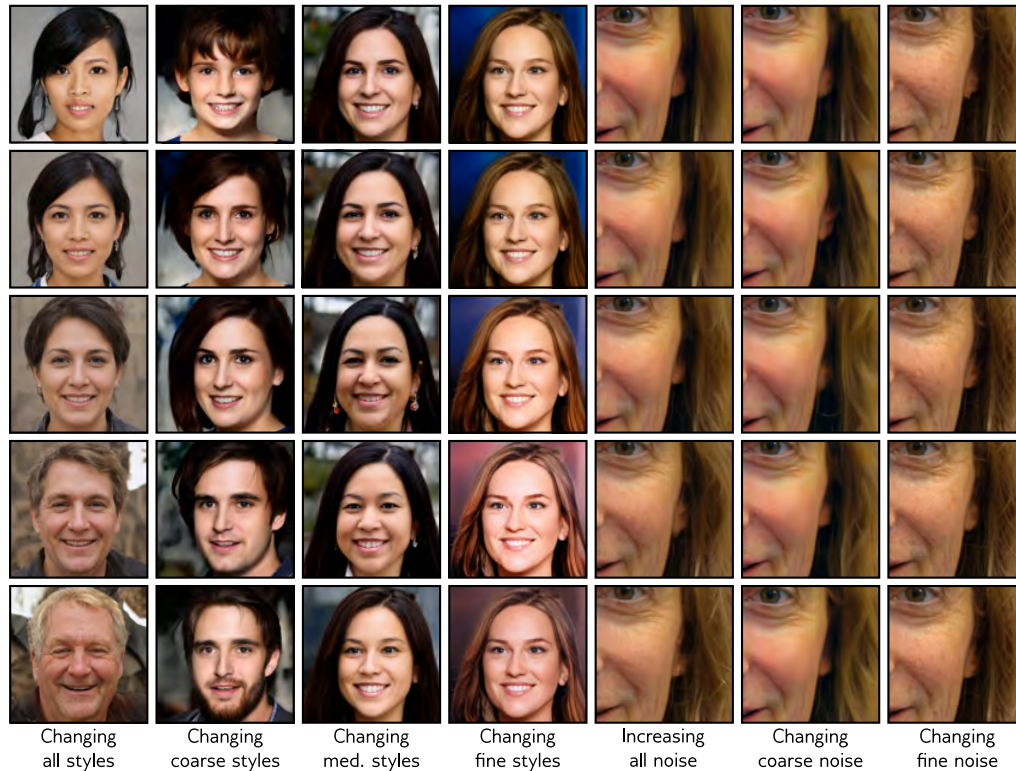


Figure 15.20 StyleGAN results. First four columns show systematic changes in style at various scales. Fifth column shows the effect of increasing noise magnitude. Last two columns show different noise vectors at two different scales.

Wasserstein GAN, which provides a more consistent training signal.

We reviewed convolutional GANs for generating images and a series of tricks that improve the quality of the generated images, including progressive growing, mini-batch discrimination, and truncation. Conditional GAN architectures introduce an auxiliary vector that allows control over the output (e.g., the choice of object class). Image translation tasks retain this conditional information in the form of an image but dispense with the random noise. The GAN discriminator now works as an additional loss term that favors “realistic” looking images. Finally, we described StyleGAN, which injects noise into the generator strategically to control the style and noise at different scales.

Notes

Goodfellow et al. (2014) introduced generative adversarial networks. An early review of progress can be found in Goodfellow (2016). More recent overviews include Creswell et al. (2018) and

Gui et al. (2021). Park et al. (2021) present a review of GAN models that focuses on computer vision applications. Hindupur (2022) maintains a list of named GAN models (numbering 501 at the time of writing) from ABC-GAN (Susmelj et al., 2017) right through to ZipNet-GAN (Zhang et al., 2017b). Odena (2019) lists open problems concerning GANs.

Data: GANs have primarily been developed for image data. Examples include the deep convolutional GAN (Radford et al., 2015), progressive GAN (Karras et al., 2018), and StyleGAN (Karras et al., 2019) models presented in this chapter. For this reason, most GANs are based on convolutional layers, although more recently, GANs that exploit transformers in the generator and discriminator to capture long-range correlations have been developed (e.g., SAGAN, Zhang et al., 2019b). However, GANs have also been used to generate molecular graphs (De Cao & Kipf, 2018), voice data (Saito et al., 2017; Donahue et al., 2018b; Kaneko & Kameoka, 2017; Fang et al., 2018), EEG data (Hartmann et al., 2018), text (Lin et al., 2017a; Fedus et al., 2018), music (Mogren, 2016; Guimaraes et al., 2017; Yu et al., 2017), 3D models (Wu et al., 2016), DNA (Killoran et al., 2017), and video data (Vondrick et al., 2016; Wang et al., 2018a).

GAN loss functions: It was originally claimed that GANs converged to Nash equilibria during training. However, more recent evidence suggests that this isn't always the case (Farnia & Ozdaglar, 2020; Jin et al., 2020; Berard et al., 2019). (Arjovsky et al., 2017; Metz et al., 2017; Qi, 2020) identified that the original GAN loss function was unstable, and this led to different formulations. Mao et al. (2017) introduced the least squares GAN. For some parameter choices, this implicitly minimizes the Pearson χ^2 divergence. Nowozin et al. (2016) argue that the Jensen-Shannon divergence is a special case of a larger family of f-divergences and show that any f-divergence can be used for training GANs. Jolicoeur-Martineau (2019) introduces the relativistic GAN in which the discriminator estimates the probability that a real data example is more realistic than a generated one rather than the absolute probability that it is real. Zhao et al. (2017a) reformulate the GAN into a general energy-based framework in which the discriminator is a function that attributes low energies to real data and higher energies elsewhere. As an example, they use an autoencoder and base the energy on reconstruction error.

Arjovsky & Bottou (2017) analyzed vanishing gradients in GANs, and this led to the Wasserstein GAN (Arjovsky et al., 2017), which is based on earth mover's distance/optimal transport. The Wasserstein formulation requires that the Lipschitz constant of the discriminator is less than one; the original paper proposed to clip the weights in the discriminator, but subsequent work imposed a gradient penalty (Gulrajani et al., 2016) or applied spectral normalization (Miyato et al., 2018) to limit the Lipschitz constant. Other variations of the Wasserstein GAN were introduced by Wu et al. (2018a), Bellemare et al. (2017b), and Adler & Lunz (2018). Hermann (2017) presents an excellent blog post discussing duality and the Wasserstein GAN. For more information about optimal transport, consult the book by Peyré et al. (2019). Lucic et al. (2018) present an empirical comparison of GAN loss functions of the time.

Tricks for training GANs: Many heuristics improve the stability of training GANs and the quality of the final results. Marchesi (2017) first used the truncation trick (figure 15.10) to trade off the variability of GAN outputs relative to their quality. This was also proposed by Pieters & Wiering (2018) and Brock et al. (2019), who added a regularizer that encourages the weight matrices in the generator to be orthogonal. This means that truncating the latent variable has a closer relationship to truncating the output variance and improves sample quality.

Other tricks include only using the gradients from the top K most realistic images (Sinha et al., 2020), label smoothing in the discriminator (Salimans et al., 2016), updating the discriminator using a history of generated images rather than the ones produced by the latest generator to avoid model "oscillation" (Salimans et al., 2016), and adding noise to the discriminator input (Arjovsky & Bottou, 2017). Kurach et al. (2019) present an overview of normalization and regularization in GANs. Chintala et al. (2020) provide further suggestions for training GANs.

Sample diversity: The original GAN paper (Goodfellow et al., 2014) argued that given enough capacity, training samples, and computation time, a GAN can learn to minimize the Jensen-Shannon divergence between the generated samples and the true distribution. However, subsequent work has cast doubt on whether this happens in practice. Arora et al. (2017) suggest that the finite capacity of the discriminator means that the GAN training objective can approach its optimum value even when the variation in the output distribution is limited. Wu et al. (2017) approximated the log-likelihoods of the distributions produced by GANs using annealed importance sampling and found a mismatch between the generated and real distributions. Arora & Zhang (2017) ask human observers to identify GAN samples that are (near-)duplicates and infer the diversity of images from the frequency of these duplicates. They found that for DCGAN, a duplicate occurs with probability $>50\%$ with 400 samples; this implies that the support size was $\sim 400,000$, which is smaller than the training set. They also showed that the diversity increased as a function of the discriminator size. Bau et al. (2019) take a different approach and investigate the parts of the data space that GANs *cannot* generate.

Increasing diversity and preventing mode collapse: The extreme case of lack of diversity is *mode collapse*, in which the network repeatedly produces the same image (Salimans et al., 2016). This is a particular problem for conditional GANs, where the latent variable is sometimes completely ignored, and the output depends only on the conditional information. Mao et al. (2019) introduce a regularization term to help prevent mode collapse in conditional GANs, which maximizes the ratio of the distance between generated images with respect to the corresponding latent variables and hence encourages diversity in the outputs. Other work that aims to reduce mode collapse includes VEEGAN (Srivastava et al., 2017), which introduces a reconstruction network that maps the generated image back to the original noise and hence discourages many-to-one mappings from noise to images.

Salimans et al. (2016) suggested computing statistics across the mini-batch and using the discriminator to ensure that these are indistinguishable from the statistics of batches of real images. This is known as *mini-batch discrimination* and is implemented by adding a layer toward the end of the discriminator that learns a tensor for each image that captures the statistics of the batch. This was simplified by Karras et al. (2018), who computed a standard deviation for each feature in each spatial location over the mini-batch. Then they average over spatial locations and features to get a single estimate. This is replicated to get a single feature map, which is appended to a layer near the end of the discriminator network. Lin et al. (2018) pass concatenated (real or generated) samples to the discriminator and provide a theoretical analysis of how presenting multiple samples to the discriminator increases diversity. MAD-GAN (Ghosh et al., 2018) increases the diversity of GAN samples by using multiple generators and requiring the single discriminator to identify which generator created the samples, thus providing a signal to help push the generators to create different samples from one another.

Multiple scales: Wang et al. (2018b) used multiple discriminators at different scales to help ensure that image quality is high in all frequency bands. Other work defined both generators and discriminators at different resolutions (Denton et al., 2015; Zhang et al., 2017d; Huang et al., 2017c). Karras et al. (2018) introduced the progressive growing method (figure 15.9), which is somewhat simpler and faster to train.

StyleGAN: Karras et al. (2019) introduced the StyleGAN framework (section 15.6). In subsequent work (Karras et al., 2020b), they improved the quality of generated images by (i) redesigning the normalization layers in the generator to remove “water droplet” artifacts and (ii) reducing artifacts where fine details do not follow the coarse details by changing the progressive growing framework. Further improvements include developing methods to train GANs with limited data (Karras et al., 2020a) and fixing aliasing artifacts (Karras et al., 2021). A large body of work finds and manipulates the latent variables in the StyleGAN to edit images (e.g., Abdal et al., 2021; Collins et al., 2020; Härkönen et al., 2020; Patashnik et al., 2021; Shen et al., 2020b; Tewari et al., 2020; Wu et al., 2021; Roich et al., 2022).

Conditional GANs: The conditional GAN was developed by Mirza & Osindero (2014), the auxiliary classifier GAN by Odena et al. (2017), and the InfoGAN by Chen et al. (2016b). The discriminators of these models usually append the conditional information to the discriminator input (Mirza & Osindero, 2014; Denton et al., 2015; Saito et al., 2017) or to an intermediate hidden layer in the discriminator (Reed et al., 2016a; Zhang et al., 2017d; Perarnau et al., 2016). However, Miyato & Koyama (2018) experimented with taking the inner product between embedded conditional information with a layer of the discriminator, motivated by the role of the class information in the underlying probabilistic model. Images generated by GANs have variously been conditioned on classes (Odena et al., 2017), input text (Reed et al., 2016a; Zhang et al., 2017d), attributes (Yan et al., 2016; Donahue et al., 2018a; Xiao et al., 2018b), bounding boxes and keypoints (Reed et al., 2016b), and images (Isola et al., 2017).

Image translation: Isola et al. (2017) developed the Pix2Pix algorithm (figure 15.16), and a similar system with higher-resolution results was subsequently developed by Wang et al. (2018b). StarGAN (Choi et al., 2018) performs image-to-image translation across multiple domains using only a single model. The idea of cycle consistency loss was introduced by Zhou et al. (2016b) in DiscoGAN and Zhu et al. (2017) in CycleGAN (figure 15.18).

Adversarial loss: In many image translation tasks, there is no “generator”; these can be considered supervised learning tasks with an adversarial loss that encourages realism. The super-resolution algorithm of Ledig et al. (2017) is a good example of this (figure 15.17). Esser et al. (2021) used an autoencoder with an adversarial loss. This network takes an image, reduces the representation size to create a “bottleneck,” and then reconstructs the image from this reduced data space. In practice, the architecture is similar to encoder-decoder networks (e.g., figure 10.19). After training, the autoencoder reproduces something that is both close to the image and looks highly realistic. They vector quantize (discretize) the bottleneck of the autoencoder and then learn a probability distribution over the discrete variables using a transformer decoder. By sampling from this transformer decoder, they can produce extremely large high-quality images.

Inverting GANs: One way to edit real images is to project them to the latent space, manipulate the latent variable, and then re-project them to image space. This process is known as *resynthesis*. Unfortunately, GANs only map from the latent variable to the observed data, not vice versa. This has led to methods to *invert* GANs (i.e., find the latent variable that corresponds as closely as possible to an observed image). These methods fall into two classes. The first learns a network that maps in the opposite direction (Donahue et al., 2018b; Luo et al., 2017a; Perarnau et al., 2016; Dumoulin et al., 2017; Guan et al., 2020). This is known as an *encoder*. The second approach is to start with some latent variable $\mathbf{z}$ and optimize it until it reconstructs the image as closely as possible (Creswell & Bharath, 2018; Karras et al., 2020b; Abdal et al., 2019; Lipton & Tripathi, 2017). Zhu et al. (2020a) combine both approaches.

There has been particular interest in inversion for StyleGAN because it produces excellent results and can control the image at different scales. Unfortunately, Abdal et al. (2020) showed that it is not possible to invert StyleGAN without artifacts and proposed inverting to an extended style space, and Richardson et al. (2021) trained an encoder that reliably maps to this space. Even after inverting to the extended space, editing images that are out of domain may still not work well. Roich et al. (2022) address this issue by fine-tuning the generator of StyleGAN so that it reconstructs the image exactly and show that the result can be edited well. They also add extra terms that reconstruct nearby points exactly so that the modification is local. This technique is known as *pivotal tuning*. A survey of GAN inversion techniques can be found in Xia et al. (2022).

Editing images with GANs: The iGAN (Zhu et al., 2016) allows users to make interactive edits by scribbling or warping parts of an existing image. The tool then adjusts the output

image to be both realistic and to fit these new constraints. It does this by finding a latent vector that produces an image that is similar to the edited image and obeys the edge map of any added lines. It is typical also to add a mask so that only parts of the image close to the edits are changed. EditGAN (Ling et al., 2021) jointly models images and their semantic segmentation masks and allows edits to that mask.

Problems

Problem 15.1 What will the loss be in equation 15.9 when $Pr(\mathbf{x}^*) = Pr(\mathbf{x})$?

Problem 15.2* Write an equation relating the loss L in equation 15.8 to the Jensen-Shannon distance $D_{JS}[Pr(\mathbf{x})^* || Pr(\mathbf{x})]$ in equation 15.9.

Problem 15.3 Consider computing the earth mover's distance using linear programming in the primal form. The discrete distributions $Pr(x=i)$ and $q(x=j)$ are defined on $x = 1, 2, 3, 4$ and:

$$\mathbf{b} = [Pr(x=1), Pr(x=2), Pr(x=3), Pr(x=4), q(x=1), q(x=2), q(x=3), q(x=4)]^T. \quad (15.18)$$

Write out the contents of the 8×16 matrix $\mathbf{A}$. You may assume that the contents of $\mathbf{P}$ have been vectorized into $\mathbf{p}$ column-first.

Problem 15.4* Calculate (i) the KL divergence, (ii) the reverse KL divergence, (iii) the Jensen-Shannon divergence, and (iv) the Wasserstein distance between the distributions:

$$Pr(z) = \begin{cases} 0 & z < 0 \\ 1 & 0 \leq z \leq 1 \\ 0 & z > 1 \end{cases}, \quad \text{and} \quad Pr(z) = \begin{cases} 0 & z < a \\ 1 & a \leq z \leq a+1 \\ 0 & z > a \end{cases}. \quad (15.19)$$

for the range $a \in [-3, 3]$. To get a formula for the Wasserstein distance for this special case, consider the total “earth” (i.e., probability mass) that must be moved and multiply this by the distance it must move.

Problem 15.5 The KL distance and Wasserstein distances between univariate Gaussian distributions are given by:

$$D_{kl} = \log \left[\frac{\sigma_2}{\sigma_1} \right] + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}, \quad (15.20)$$

and

$$D_w = (\mu_1 - \mu_2)^2 + \sigma_1 + \sigma_2 - 2\sqrt{\sigma_1\sigma_2}, \quad (15.21)$$

respectively. Plot these distances as a function of $\mu_1 - \mu_2$ for the case when $\sigma_1 = \sigma_2 = 1$.

Problem 15.6 Consider a latent variable $\mathbf{z} \sim \text{Norm}[\mathbf{0}, \mathbf{I}]$ with dimension 100. Consider truncating the values of this variable to (i) $\tau = 2.0$, (ii) $\tau = 1.0$, (iii) $\tau = 0.5$, (iv) $\tau = 0.04$ standard deviations. What proportion of the original probability distribution is disregarded in each case?